SBC Vision

Note de cadrage | Sujet 2 : vision pour piloter une interface

Groupe et répartition du travail

1. **Saber Kasri - Données et vision :** captures réelles, annotations, entraînement et évaluation du détecteur.
2. **Rayan Mazri - OCR et optimisation :** lecture des cartes, inventaire, solveur SBC et données FUTBIN.
3. **Samy Allali - Application et intégration :** interface, contrôleur visuel, API, cloud et démonstrateur de secours.

Sujet choisi et objectif

Nous choisissons le **sujet 2 : vision pour piloter une interface**, appliqué aux défis de création d'équipe (SBC) de la **vraie Web App EA FC 27**. Son interface dense, ses cartes visuellement proches et ses interactions répétitives motivent ce choix. La réussite est mesurable : actions identifiées, équipe valide et coût maîtrisé. La vision constitue le cœur du projet ; le solveur en est l'application métier.

Solution et parcours utilisateur

Une application web accompagne la préparation du SBC ; un contrôleur local observe le navigateur et exécute les actions prévues.

1. **Configurer.** L'utilisateur ouvre sa session, choisit un SBC, fixe un budget et protège les cartes à conserver.
2. **Inventorier.** Le contrôleur parcourt les pages et fiches, puis capture automatiquement les écrans. Détection et OCR lisent les cartes et les conditions ; les versions et quantités sont sauvegardées, les ambiguïtés signalées.
3. **Optimiser.** Un solveur de contraintes choisit une équipe valide en minimisant les achats et la valeur sacrifiée. Il exclut les cartes protégées et signale l'absence de solution.
4. **Recommander.** Si un achat réduit le coût global, l'application suggère directement des joueurs : version exacte, poste, prix et plafond d'achat, avec des alternatives compatibles avec toute l'équipe.
5. **Placer et contrôler.** Le contrôleur recherche et place les cartes aux coordonnées détectées, puis vérifie l'écran obtenu. Un récapitulatif présente les cartes utilisées et le coût avant confirmation explicite de l'utilisateur.

Fonctionnement et données

Entrée : capture d'écran. **Sortie :** JSON des éléments visibles (type, texte, coordonnées, confiance, actions). L'agent est un **contrôleur scripté** consommant ce JSON pour décider des clics et saisies ; détection et OCR structurent l'inventaire et les conditions. L'application React appelle en HTTPS une API Python FastAPI ; le modèle reste côté serveur, conteneurisé avec Docker et déployé dans le cloud. Stack : OpenCV, YOLO affiné, OCR, OR-Tools, SQLite ; dépôt Epitech.

Données métier : catalogue FUTBIN importé par lots JSON en SQLite ; prix actualisés de façon ciblée, sourcés et datés. Accès testé ; attribution à FUTBIN pour l'usage étudiant des prix (CGU, section 13).

Données de vision et évaluation

Cible initiale : **300 à 500 captures annotées**, comprenant des captures de la vraie Web App collectées lors de navigations manuelles (club, fiches, SBC), et un sous-ensemble de **WebUI** aux classes harmonisées. Les sessions EA FC sont séparées entre entraînement et test. Pour la généralisation, au moins **50 captures de 5 domaines web distincts** (boutiques, réservation, tableaux de bord) sont entièrement exclues de l'entraînement et de la validation.

Comparaison sur les mêmes données : **OpenCV + OCR** et **YOLO affiné + OCR**, avec une troisième référence **OmniParser sans adaptation** si les ressources le permettent. Mesures : mAP@0,5 globale et par domaine, erreur OCR, F1 des éléments JSON (type, action, localisation), validité du schéma et latence. Les entraînements sont tracés dans MLflow ; les erreurs et le coût des compositions sont analysés.

Périmètre et validation demandée

Livraison en décembre : un SBC à composition d'équipe, un club limité et des règles documentées. **La cible reste la vraie Web App** ; son automatisation dépend des autorisations d'EA. À défaut, Samy réalisera un **démonstrateur local minimal** des écrans nécessaires, avec soumission simulée ; la vision restera évaluée sur des captures réelles. Le cas d'usage, la généralisation et ce repli sont soumis au superviseur.

Références : github.com/js0nwu/webui ; github.com/microsoft/OmniParser ; futbin.com/tos (section 13).